

# BSDA2001P: Introduction to DL and GenAI Project

## Project Report

### 1. Abstract / Executive Summary

This project is about solving the Smart MCQ Solver Challenge, where the goal is to predict the correct answer for multiple-choice questions and get a good MAP@3 score.

I used three models: a custom neural network built from scratch using PyTorch, a fine-tuned RoBERTa model, and a Sentence Transformer with cosine similarity. I compared the models using Accuracy, F1-score, and MAP@3, tracked the results using Weights & Biases, and got a final Kaggle score of 0.74.

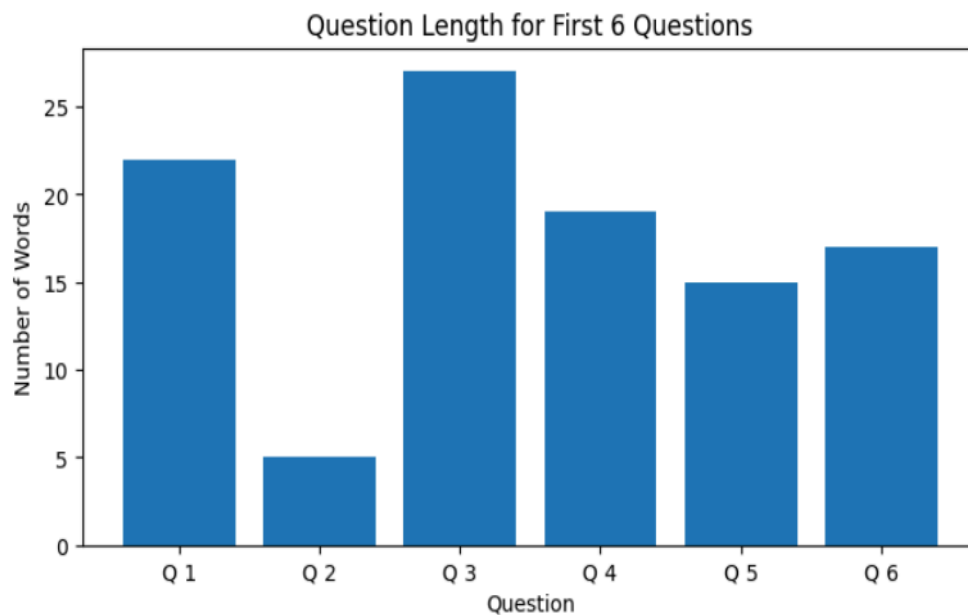
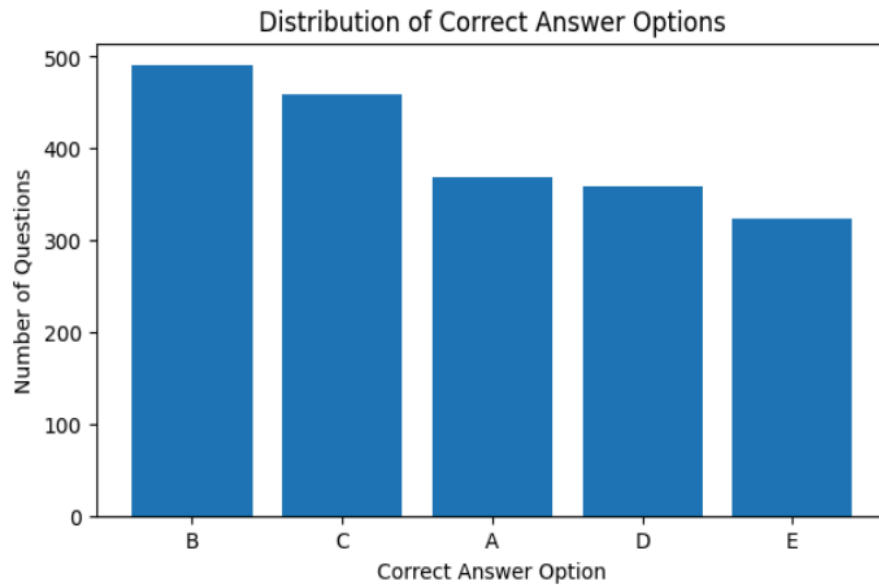
### 2. Introduction

- **Problem Statement:** The goal of this project is to solve the Smart MCQ Solver Challenge by predicting the correct answer for each question. The model should rank the correct answer as high as possible
- **Project Objective:** The main objective of this project is to build and compare different deep learning models.
- **Report Structure:** This report explains the dataset, preprocessing steps, tokenization, model implementation, and evaluation process. It also compares the performance of the three models, discusses the final results, and summarizes the key learnings.

### 3. Dataset & Preprocessing

- **Dataset Description:** The dataset used in this project is from the Smart MCQ Solver Challenge on Kaggle. It contains multiple-choice questions, five answer options (A, B, C, D, E), and the correct answer for each question.
- **Exploratory Data Analysis (EDA):** The dataset was checked to understand its structure and answer distribution before training the models. I looked at the text data to make sure the dataset was ready for preprocessing. Basic visualizations were also created to understand patterns in the dataset, such as the distribution of correct answer options and other relevant data characteristics.

# BSDA2001P: Introduction to DL and GenAI Project



- **Data Preprocessing:** The text was cleaned by converting it to lowercase, removing special characters, numbers, and extra spaces. It was also checked for null values but there were no nulls present.

# BSDA2001P: Introduction to DL and GenAI Project

## 4. Tokenization Strategy

- **Primary Tokenizer Selection:** For the RoBERTa model, I used the RoBERTa tokenizer provided by the Hugging Face Transformers library. It converts the text into tokens that the pretrained RoBERTa model can understand and process.
- **Implementation Details:** I used the Hugging Face tokenizer to tokenize the text and applied padding and truncation with a maximum sequence length of 256. This made sure that all input sequences had the same length before training the model.
- **Experimental Comparison:** The scratch model used a custom tokenizer and vocabulary, the RoBERTa model used the RoBERTa tokenizer, and the Sentence Transformer used its own pretrained tokenizer.

## 5. Modeling & Experimentation

### 5.1. Model 1: Fine-Tuned Transformer Model

- **Architecture:** I used the RoBERTa-base pretrained transformer model from the Hugging Face Transformers library. I fine-tuned the model on the MCQ dataset by treating each question-option pair as a binary classification problem.

<https://huggingface.co/roberta-base>

- **Salient Points:** RoBERTa uses the self-attention mechanism to understand the relationship between words in a sentence. This helps the model understand the meaning of the question and each answer option.
- **Fine-Tuning Strategy:** I fine-tuned the model for 4 epochs, a learning rate of  $2e-5$ , and a batch size of 16. The best model was selected based on the validation performance, and no layers were frozen during training.

### 5.2. Model 2: Custom Deep Learning Model

- **Architecture:** I built a custom neural network from scratch using PyTorch. The model consists of an Embedding layer, Average Pooling, Fully Connected layers, and an Output layer for binary classification. [Architectural Diagram](#)
- **Salient Points:** The model learns word embeddings from scratch without using any pretrained model. It uses average pooling to combine the word embeddings and fully connected layers to predict whether a question-option pair is correct or incorrect.

# BSDA2001P: Introduction to DL and GenAI Project

## 5.3. Model 3: Additional Model of Choice

- **Architecture:** I used the pretrained all-MiniLM-L6-v2 Sentence Transformer model to generate embeddings for the question and answer options. Cosine similarity was then used to compare the embeddings and rank the answer options.
- **Salient Points:** Instead of classifying each option directly, this model measures how similar each answer option is to the question. The options are ranked based on cosine similarity, and the top three answers are used to calculate the MAP@3 score.

## 6. Performance & Comparative Analysis

- **Evaluation Metrics:** The models were evaluated using Accuracy, F1-score, and MAP@3. Accuracy and F1-score measure the classification performance, while MAP@3 is the main evaluation metric used in the Kaggle competition.
- **Training Performance:** The performance of all models was tracked using Weights & Biases (W&B). The training results, including Accuracy, F1-score, and MAP@3, were used to compare the models.



- **Comparative Report:** The RoBERTa model performed the best among the three models. The Sentence Transformer gave the lowest scores among the three models.

# BSDA2001P: Introduction to DL and GenAI Project

| Model                | MAP@3 score | Accuracy score | F1 score |
|----------------------|-------------|----------------|----------|
| Scratch Model (NN)   | 0.929       | 0.877          | 0.875    |
| RoBERTa              | 0.984       | 0.951          | 0.870    |
| Sentence Transformer | 0.382       | 0.225          | 0.224    |

- **Kaggle Performance:** The submission achieved a score of 0.748 on the Kaggle leaderboard, with a rank of 874

## 7. Conclusion & Future Work

- **Key Learnings:** This project helped me understand how different deep learning models can be used for an NLP task. I learned how to build a custom neural network, fine-tune a pretrained transformer model and use Sentence Transformers for semantic similarity.
- **Challenges Faced:** Some of the challenges I faced were creating a model from scratch and improving the Kaggle score.
- **Areas for Improvement:** This project can be improved by using larger pretrained transformer models with LoRA fine-tuning to improve performance while reducing training cost. An RNN-based model such as LSTM or GRU can also be implemented and compared, since it was not used in this project.

## 8. References

Hugging Face. *RoBERTa-base Model Card*: <https://huggingface.co/roberta-base>

Hugging Face. *all-MiniLM-L6-v2 Model Card*:  
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>